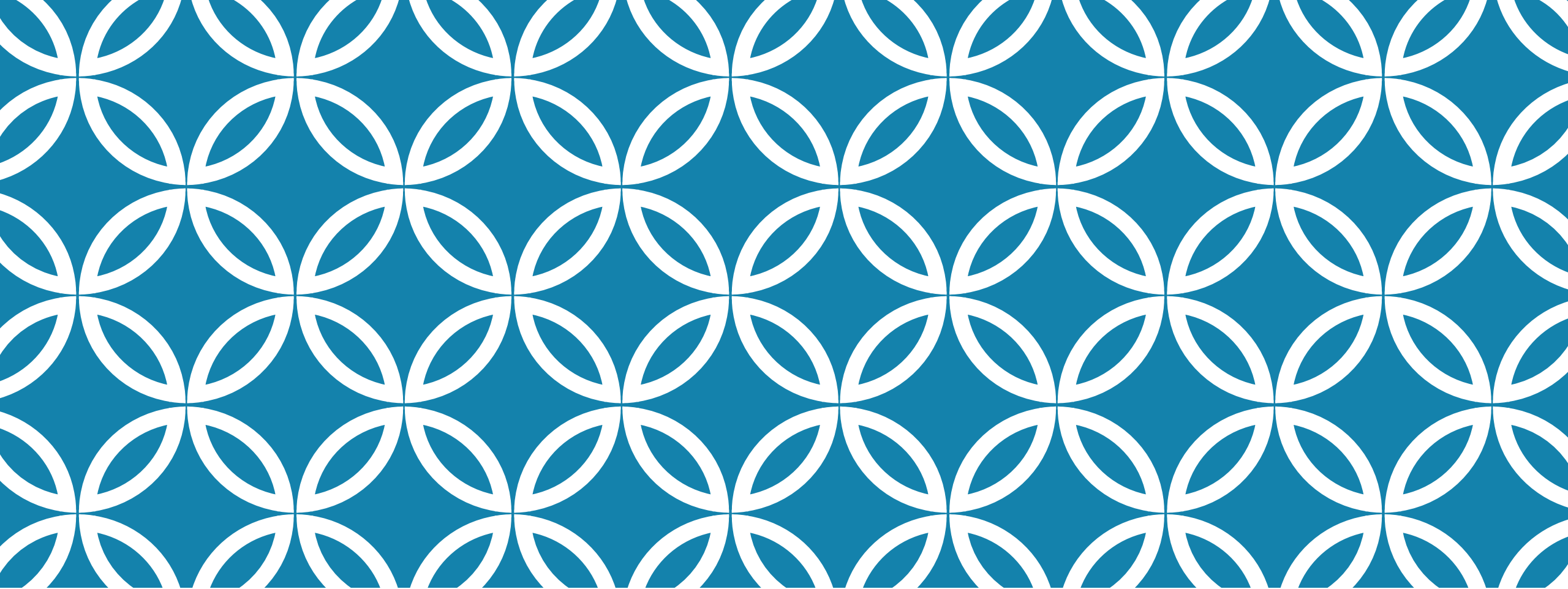




COMP 52

DASD & TAPE

COMP 52 Module 8



DIRECT ACCESS STORAGE DEVICES (DASD)

UNIT parameter

requests device e.g disk drive

UNIT= device-type or UNIT=group-name

e.g. UNIT=3380 ←= device type (DASD)

e.g. UNIT=SYSDA ←= group name (similar to a variable,
i.e. SYSDA may be assigned 3380)

better since device type could change
when hardware updated, group names
decided at SYSGEN and assigned types
SYSDA=**S**ystem **D**irect **A**ccess

Picture shows many disk drives



VOL=SER= parameter:

Specifies which volume you want to allocate for your data set.

If omitted, MVS scans for eligible volumes based on the UNIT parameter.

e.g. VOL=SER=SLC123

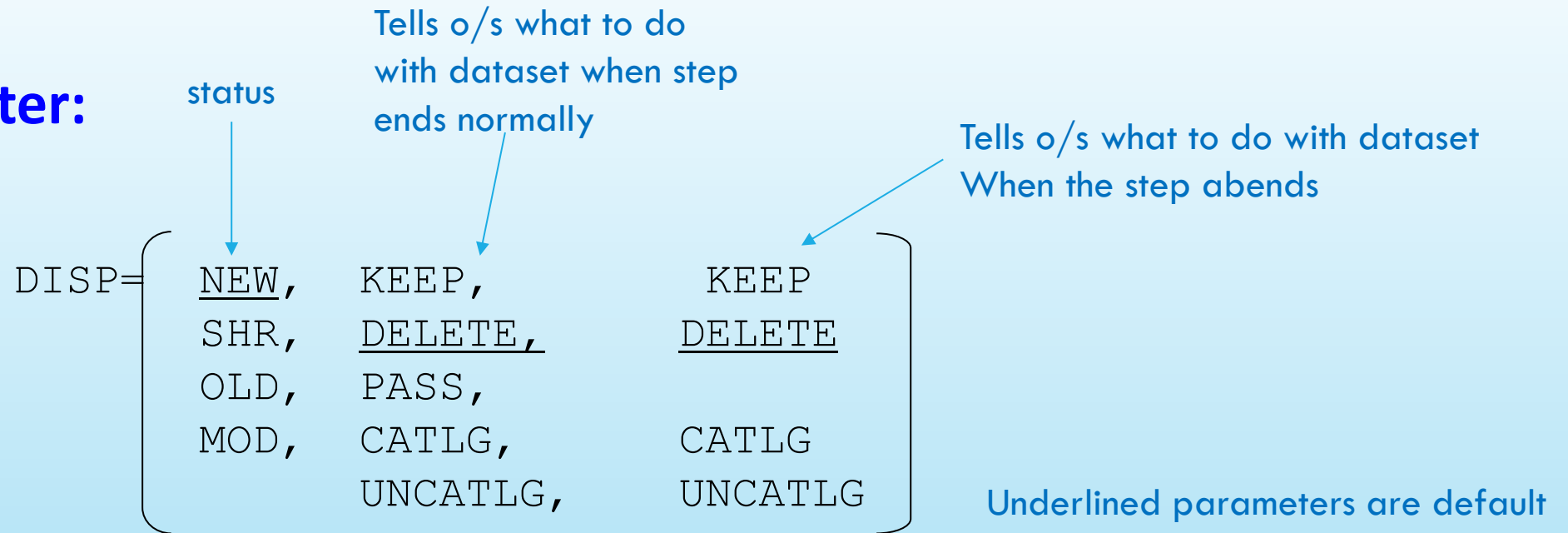


Each diskpack has its own unique serial number.

A copy of the serial number appears on the outside of the diskpack

Note: today most diskpacks are virtual, but the parameters remain the same.

DISP Parameter:



These are positional subparameters.

Need a comma if omitting a subparameter before other subparameters.

If the DISP parameter is omitted, the default is: DISP=(NEW,DELETE,DELETE)

This is used for a temporary data set as it is created and deleted in the same program.

Recommendation: Always code a DISP parameter when coding a DSN

CATLG

- An entry is placed in a system catalog that saves the location (UNIT and VOL) so that when accessing the data set, you do not need to code UNIT and VOL parameters
- NEW data sets are automatically saved on the system catalogue
- If a data set is not used frequently, it can be removed from the catalogue
- Cannot catalogue temporary files

e.g. DISP=(NEW,CATLG,DELETE) is used to create a dataset as well as catalogue it

e.g. //ddname DD DSN=MYFILE, is used to catalogue a data set that already exists, but is not catalogued

```
//          DISP=(SHR/OLD,CATLG,KEEP),  
//          UNIT=SYSDA,  
//          VOL=SER=STPK01
```

therefore if when accessing the data set from now on:

```
//ddname DD DSN=MYFILE,  
//          DISP=(SHR,KEEP,KEEP)
```

To delete it: //ddname DD DSN=MYFILE,
// DISP=(OLD,DELETE,KEEP)

therefore it removes it from both the VTOC and the catalog

DISP parameter default values

status If omitted, assumes that a data set is being created. e.g. DISP=(,KEEP,KEEP)
if the dataset already exists, error message: Duplicate data set on volume

normal disposition Default depends on the value specified or assumed for status:
if NEW, normal disposition is DELETE. e.g. DISP=(NEW) → DISP=(NEW,DELETE)
if OLD, SHR, or MOD, normal disposition is KEEP. e.g. DISP=(OLD) → DISP=(OLD,KEEP)

abnormal disposition Indicates what to do with data set if the jobstep abends
default: Takes on the value specified or assumed for normal disposition, unless PASS.

e.g. DISP=(,DELETE) → DISP=(NEW,DELETE,DELETE)
DISP=SHR → DISP=(SHR,KEEP,KEEP)

UNCATLG keeps the data set, just removes it from the catalogue

```
e.g. //ddname DD DSN=MYFILE,  
      //      DISP=(SHR,UNCATLG,KEEP)
```

```
careful: //ddname DD DSN=MYFILE,  
          //      DISP=(OLD,DELETE,KEEP),  
          //      UNIT=SYSDA,  
          //      VOL=SER=STPK01
```

removes from VTOC only (O/S is unaware the dataset might be catalogued), if its catalogued it will remain on catalogue but off the VTOC, therefore would need to uncatalogue it

PASS next week

- useful for multistep jobs, when a data set is used more than once
- tells MVS to place info. about the data set in a pass list, therefore subsequent steps that access the data set don't have to code UNIT and VOL
- can pass permanent or temporary data sets
- saves time since system keeps the pass list in memory, therefore don't have to read VTOC

e.g. rewinds tapes and positions at start, ready for next jobstep to access it

- pass catalogued data sets to save time (faster retrieval from a pass list than in a catalogue)
- if data set is passed at end of a job, new datasets are deleted and existing ones are kept
- can't code pass for abnormal

Temporary Data Sets

Many jobs require temporary data sets that exist only for the duration of a job or a job step.

Format: DSN=&&tempname

- Must be 1 to 8 alphanumeric or national characters, starting with an alphabetic or national character.
- Must be preceded by two ampersands (&&).
- Compound names joined by periods are not allowed.
- dataset name must be unique per JOB.
- cannot be saved

e.g. //CREATE DD DSN=&&TEMPFILE,
// DISP=(NEW,DELETE),
// DCB=(LRECL=80,BLKSIZE=800,RECFM=FB),
// SPACE=(TRK,(2,1))

Always specify the data set will be deleted at last use to release space used by data set

e.g. //NEW DD DSN=&&TEMP,
// DISP=(NEW,PASS),
// DCB=(LRECL=100,BLKSIZE=500,RECFM=FB),
// SPACE=(500,(50,1))
// :
//READ DD DSN=&&TEMP,
// DISP=(OLD,DELETE)

PASS indicates file is used again in job

Don't code UNIT/VOL for passed files

VIO temporary data sets

- Virtual Input/Output, or just VIO appears as a standard DASD data set to a program, but it doesn't reside on DASD.
- VIO data sets reside in virtual storage, therefore access is faster than a standard DASD data set.
- VIO is used only for temporary data sets.
- Use the group name VIO on the UNIT parameter: UNIT=VIO

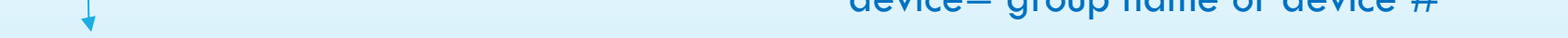
Advantages of using VIO

- VIO data sets reside in virtual storage, so reading from and writing to them is performed at the speed of virtual storage access rather than at the speed of an I/O device.
- The VIO data set doesn't occupy space in the program's address space.

```
e.g. //TEMP1    DD DSN=&&TEMPFILE,  
      //        DISP=(NEW,DELETE),  
      //        UNIT=VIO,  
      //        DCB=(LRECEL=100,BLKSIZE=500,RECFM=FB),  
      //        SPACE=(TRK,(1,1))
```

Note: VIO data sets can be passed i.e. DISP=(NEW,PASS).

Multi-volume DASD data sets

Format: 

UNIT=(device,unit-count), device= group name or device #
unit-count = # volumes

VOL=SER=(serial-number,serial-number...) Serial # =serial # for each volume required.
Not required for catalogued files

- For uncatalogued data sets, list all of the serial#'s in the VOL parameter.
- When creating a cataloged multi-volume data set, no VOL parameter is required, code a unit count in the UNIT parameter indicating the number of volumes required.
- Always CATLG files if records will be added, to ensure the data set is extended onto a new volume.
- When you create a multi-volume data set, the primary space you specify in the SPACE parameter is allocated on the first volume that's listed in the VOLUME parameter or chosen by the system. When a total of 16 extents have been allocated on the first volume, or when the diskpack is full the secondary extent is used for remaining next volumes.

Multi-volume DASD data sets Examples

```
e.g.//CUSTMAST DD DSN=CUSTOMER.MASTER.FILE,  
//              DISP=(NEW,KEEP),  
//              UNIT=SYSDA,  
//              VOL=SER=(SLC100,SLC101,SCL102),  
//              SPACE=(CYL,(40,20))
```

Creates an uncatalogued data set
specifying 3 volumes.
UNIT and VOL will need to be coded
When accessing it.

```
e.g.///CUSTMAST DD DSN=ITEM.MASTER.FILE,  
//              DISP=(NEW,CATLG),  
//              UNIT=(SYSDA,3),  
//              SPACE=(CYL,(40,20))
```

Creates a catalogued data set
Indicating that 3 volumes are required
And to be chosen by the system.
No UNIT and VOL are required
when accessing it.

SPACE parameter

directory blocks are coded for PDS files only

Format: SPACE=(unit,(primary[,secondary][,dir])[,RLSE][,{CONTIG}][,ROUND])

These are all positional sub parameters

RLSE releases any unused whole tracks if unit of measure is TRK or blocksize
releases any whole unused cylinders if unit of measure is CYL

e.g. SPACE=(TRK,(2,2),RLSE)

therefore if file requires 5 tracks, one will be released

Will release any unused whole tracks. To avoid fragmentation, don't code if the data set will have records added.

CONTIG allocates space in adjacent cylinders or tracks

The primary allocation must allocated with one extent of contiguous cylinders.

Applies only to primary, secondary can have 5 extents each

e.g. SPACE=(CYL,(2,2),,CONTIG)

Notice the extra comma since CONTIG is positional

ROUND coded only when space is requested in terms of blocks, allocates whole cylinders even though the allocation has been requested in blocks

e.g. SPACE=(400,(500,2),,,,ROUND)

Blocksize must be coded for the units, rounds up to cylinders

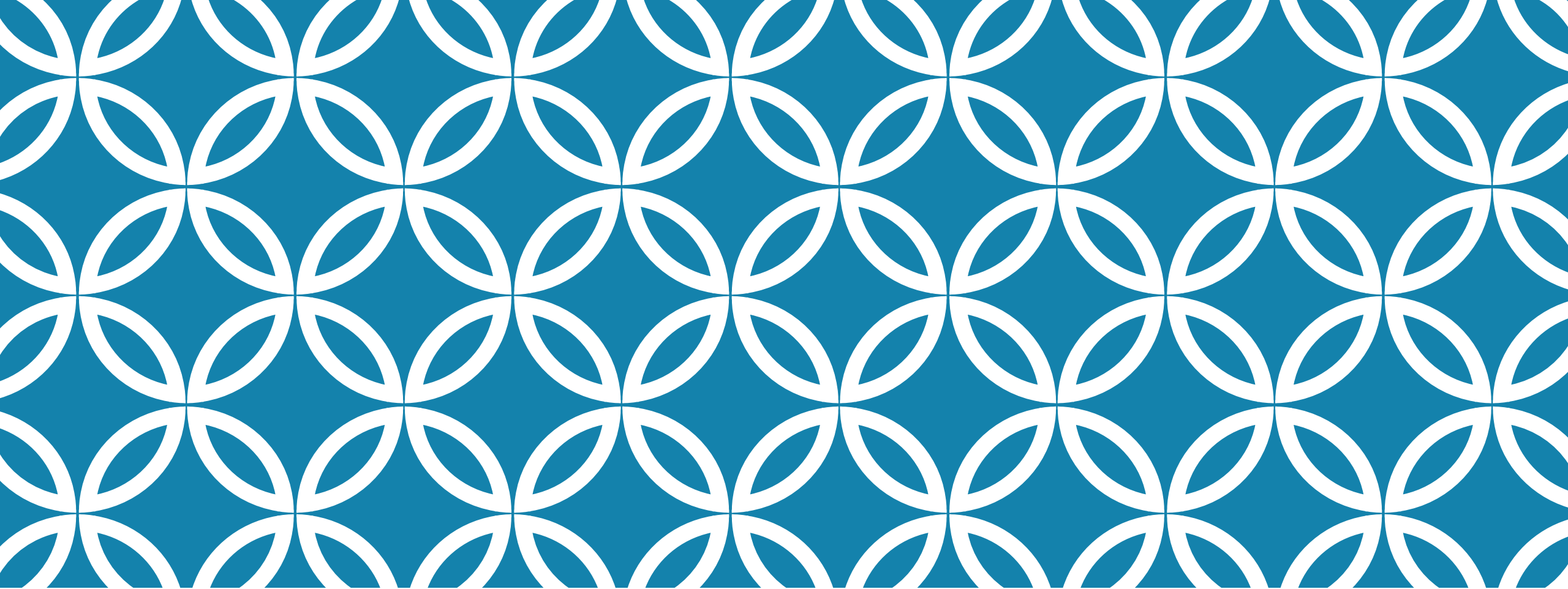
e.g. SPACE=(400,(500,2),,CONTIG,ROUND)

Concatenating data sets

You can concatenate data sets if you need to treat two or more input data sets as if they were a single file. To concatenate files, you code the ddname for the first data set, then omit the ddname for the next file. You can also concatenate data sets with different block sizes (BLKSIZE) and logical record lengths (LRECL).

e.g. **//JOB LIB DD DSN=MYLIB,
// DISP=SHR
// DD DSN=YOURLIB,
// DISP=SHR**

Searches the first PDS first



MAGNETIC TAPES/ CARTRIDGE (TAPE)

Tape Data Sets

First tapes, were reels,
Then replaced with cartridge
(similar to a VCR tape)



- used for storing large amounts of data, long term storage (archives), off-site storage
- Like DASD, tape referred to as a **volume**, and each reel or cartridge has a **serial number**
- sequential I/O only, sequential data sets (but non sequential data sets can be backed up to tapes)
- can have several data sets on one volume
- no VTOC
- operators must handle tapes, not programmers, mount tape and dismounts when done
- slow processing (human intervention required) also, if your job is assigned a tape drive, no one else can use
- place a ring in tape to write “no ringee no writee”



Unit is a Tape drive
Code UNIT=TAPE or device #

Volume is reel or cartridge
Code VOL=SER=serial #
For a specific tape

Placing a ring on the tape
allowed writing to it

Event though the technology has changed,
we still use the term tape today

e.g.

```
//JOURNAL DD DSN=SLC.JOURNAL,  
//          DISP=OLD,  
//          UNIT=TAPE  
//          VOL=SER=300123
```

Access an uncatalogued (since unit/vol coded)
tape file with serial #300123

```
//JOURNAL DD DSN=SLC.JOURNAL,  
//          DISP=(NEW,CATLG,DELETE),  
//          UNIT=TAPE,  
//          DCB=(LRECL=100,BLKSIZE=1000,RECFM=FB)
```

Create a tape file, allow O/S to select volume

NEVER CODE SPACE WITH TAPE!
if more space is needed, a new tape is added.
you don't need to reserve space like DASD

```
//JOURNAL DD DSN=SLC.JOURNAL,  
//          DISP=OLD,  
//          UNIT=TAPE,  
//          VOL=SER=(300123,300124,300125,300126)
```

therefore assigns 1 tape drive, operator asked to mount new tape when done with first volume.
Processed sequentially

e.g. UNIT=(TAPE,2)

← requests 2 tape drives. Processed sequentially on 2 tape drives

If coded with: VOL=SER=(163013,163014,163015,163016)

first 2 volumes mounted,
when first one is done, third is mounted

RETPD parameter Can be used to specify how long the data set should exist.

Prevents the file from being deleted.

RETPD=nnnn (retention period) where nnnn is 0 to 9999 days

e.g. //LNMAST DD DSN=SALES.MASTER,
 // DISP=(NEW,KEEP),
 // UNIT=TAPE,
 // VOL=SER=300123,
 // RETPD=30 ←after 30 days the data set can be deleted or written over

EXPDT parameter Ensures the data set cannot be deleted or overwritten before the date specified

EXPDT= {yyyy/ddd} Julian date.

//JOURNAL DD DSN=PRODUCT.MASTER,
// DISP=(NEW,CATLG,DELETE),
// UNIT=TAPE,
// EXPDT=2020/365 ←expiry date is set for end of year

LABEL parameter

Code the LABEL when

- 1) Reading an uncataloged data set that isn't the first one on a tape volume.
- 2) Processing a data set that doesn't have IBM standard labels. ————— These are positional subparameters

LABEL=([data-set-sequence][,label-type])

e.g. //JOURNAL DD DSN=MMA2.AR.JOURNAL,

// DISP=OLD,

Accessing the 3rd file on tape.

// UNIT=TAPE,

Note: if a data set is catalogued, then you don't need to code LABEL

// VOL=SER=300127,

// LABEL=3

← default is LABEL=1 i.e. first file on tape

If you're using IBM standard labels you don't need to code the label-type subparameter

But if the file has special labels, need to specify label type even if the data set you're accessing is cataloged

e.g. LABEL=(,SUL) ← first file on tape, Standard User labels

e.g. LABEL=(4,NSL) ← fourth file on tape, non Standard labels

Valid Labels

Code	Meaning
SL	The tape has standard IBM labels (the default).
SUL	The tape has standard IBM labels as well as user labels.
AL	The tape has ISO/ANSI/FIPS version 3 labels.
AUL	The tape has ISO/ANSI/FIPS version 3 labels and user labels.
NSL	The tape has non-standard labels.
NL	The tape has no labels.
BLP	The tape may or may not have labels, but label processing is bypassed.
LTM	Same as BLP, except that the leading tapemark on the volume is skipped.